Large Reasoning Models

Reasoning, Prompting Techniques, and Instruction Tuning

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026

Table of Contents

Motivation

- **Why study LRMs?**
  - LLMs like GPT-3 and GPT-4 excel at generating text.
  - However, they often rely on **pattern matching**, not true reasoning.
  - Real-world applications like **math problem-solving**, **code synthesis**, and **scientific hypothesis generation** require multi-step reasoning.
- **Examples:**
  - *GPT-3 can generate realistic text but fails at solving "If Alice has 3 apples and gives Bob 2, how many does she have?" unless prompted with reasoning steps.*
  - *DeepMind's AlphaCode uses reasoning techniques to solve programming problems—pure LLMs fall short.*

Learning Outcomes

By the end of this session, you should be able to:

- Define and distinguish Large Reasoning Models (LRMs) from simpler pattern-matching LLMs.
- Understand advanced reasoning techniques, including Chain-of-Thought (CoT), Tree-of-Thought (ToT), and self-consistency methods.
- Explain how instruction tuning and reinforcement learning (RL) can refine and enhance reasoning abilities in language models.
- Critically assess future directions and identify open problems in the field of reasoning with language models.

What Are Large Reasoning Models (LRMs)?

- **Definition:**
  - Large Language Models (LLMs) optimized for **multi-step logical reasoning**
  - Emphasize Chain-of-Thought (CoT) style reasoning
- **Key Examples:**
  - OpenAI: o1, o3
  - Google: Gemini 2.0 (Flash mode)
  - DeepSeek: R1, V3
  - QwQ (Alibaba)
  - Claude 3.5
  - Codestral (Mistral)

What Are Large Reasoning Models (LRMs)? (cont.)

- **Distinctive Behavior:**
  - "Thinking internally"
  - Producing hidden or explicit reasoning traces
  - Step-by-step intermediate computations
  - Intermediate reasoning traces (explicit or latent)
  - Better performance on symbolic tasks and math
  - Often paired with advanced prompting or finetuning

What Are Large Reasoning Models (LRMs)? (cont.)

- **Examples:**
  - **PaLM + CoT Prompting:**
    Google's PaLM + CoT prompt significantly improved accuracy on GSM8K (grade-school math) from approx 18% to 58%.
  - **Gemini 2.0 Flash mode:**
    Provides transparent reasoning steps for decision-making.
  - **DeepSeek R1:**
    Combines expert models for efficient coding and debugging.
    Claimed to achieve "Aha!" moments, solving reasoning tasks using structured thinking steps, not just memorization.

LLMs vs. LRMs

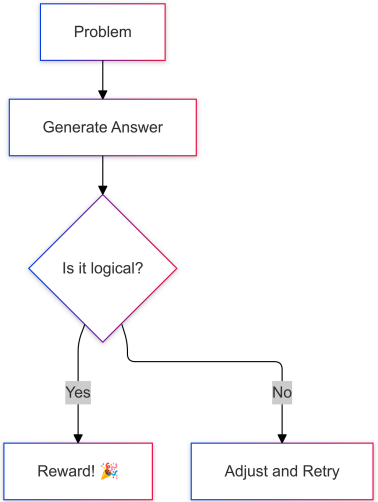
Large language models (LLMs) and large reasoning models (LRMs) share foundational technologies but serve different purposes:

- **Core Function:**
  - **LLMs:** Learn patterns in data to generate fluent, human-like text.
  - **LRMs:** Extend LLMs to solve problems requiring logical reasoning and contextual understanding.
- **Use Cases:**
  - **LLMs:** Content generation, language translation, summarization, and answering user queries.
  - **LRMs:** Math problem solving, interpreting ambiguous clinical data, and complex decision-making.

LLMs vs. LRMs (cont.)

- **Reasoning Ability:**
  - **LLMs:** Limited structured reasoning; may struggle with multi-step logic or ambiguity.
  - **LRMs:** Trained to apply consistent reasoning steps; outputs are more logical and verifiable.
- **Performance:**
  - **LLMs:** Fast response times; optimized for scalability and speed.
  - **LRMs:** Slower response times; require more processing to reason through problems step by step.
- **Best Suited For:**
  - **LLMs:** High-volume, low-risk tasks where speed is critical.
  - **LRMs:** High-stakes or complex tasks where accuracy, explainability, and logic matter.

How Do LRMs Work?



How Do LRMs Work? (cont.)

LRMs use a combination of training methods and prompt strategies to enhance the reasoning capabilities of LLMs.

- **Training on Enriched Datasets**
  - Includes reasoning-focused examples
  - Real-world scenarios with clear outcomes
  - Teaches both correct answers and reasoning steps
- **Reinforcement Learning (RL)**
  - Rewards for logical, correct answers
  - Penalties for incorrect or inconsistent reasoning
  - Reinforces desirable reasoning patterns
- **Human Feedback (RLHF)**
  - Human reviewers guide model outputs

How Do LRMs Work? (cont.)

- Refines nuanced reasoning strategies
- Useful in domains needing expertise (e.g., healthcare, finance)

- **Prompt Engineering**
  - Carefully designed prompts activate reasoning
  - Guides model through multi-step or layered tasks
  - Generic prompts may not trigger deep reasoning
- **Chain-of-Thought (CoT) Prompting**
  - Breaks problems into smaller steps
  - Explores multiple reasoning paths
  - Improves transparency and accuracy
  - Essential for auditability and explainability

Types of Reasoning in LRMs

Large reasoning models (LRMs) employ several types of reasoning to simulate or support human-like problem solving:

- **Deductive Reasoning**
  - Applies general rules to specific cases for logically certain conclusions (top-down reasoning).
  - Excels at structured, rule-based tasks requiring high logical consistency.
  - Example domains: regulatory compliance, medical protocol analysis.
  - *Example:*
    Rule: All patients with a fever and cough should be tested for influenza.
    Case: Patient A has a fever and cough.
    Deduction: Patient A should be tested for influenza.

Types of Reasoning in LRMs (cont.)

- **Inductive Reasoning**
  - Draws general conclusions from specific observations in data.
  - Supports probabilistic predictions and generalization to unseen inputs.
  - Useful in dynamic, data-rich scenarios (e.g., fraud detection).
  - *Example:*
    Observation: 90% of emails with suspicious links are phishing attempts.
    Induction: An email with a suspicious link is likely to be a phishing attempt.

Types of Reasoning in LRMs (cont.)

- **Abductive Reasoning**
  - Infers the most likely explanation from incomplete or uncertain evidence.
  - Generates hypotheses rather than definitive answers.
  - Effective in domains like medical diagnostics, where plausible interpretations are needed.
  - *Example:*
    Evidence: The lawn is wet in the morning.
    Possible explanations: It rained overnight, or the sprinkler was on.
    Abduction: If the weather report says no rain, the most likely explanation is the sprinkler was on.

Types of Reasoning in LRMs (cont.)

- **Analogical Reasoning**
  - Identifies similarities between different situations or datasets.
  - Transfers insights from one context to another for novel problem solving.
  - Example: recommending products based on similar customer behavior.
  - *Example:*
    Situation A: A student learns to solve quadratic equations by factoring.
    Situation B: The student applies the same factoring method to solve cubic equations.
    Analogy: The approach used for quadratics is adapted to cubics.

Reasoning vs. Pattern-Matching in LLMs

- **Pattern-Matching Models**
  - **How it works:** LLMs are trained on vast amounts of text data to learn statistical associations between words and phrases.
  - **Characteristics:**
    - Generate text based on learned patterns without deeper understanding.
    - Often produce fluent and coherent text.
    - May struggle with complex reasoning tasks or multi-step logic.
  - **Example:** Given the prompt "The capital of France is", the model outputs "Paris" by recalling frequent patterns in the training data.
  - **Limitations:**
    - May produce plausible-sounding but incorrect answers if the pattern is misleading.
    - Struggles with tasks that require connecting multiple facts or reasoning through steps.

Reasoning vs. Pattern-Matching in LLMs (cont.)

- **Reasoning-Focused Models**
  - **How it works:** LRMs extend LLMs by incorporating structured reasoning techniques.
  - **Characteristics:**
    - Generate structured intermediate steps to arrive at conclusions.
    - Use multi-hop inference to connect information across several statements or facts.
    - Can break down complex problems into smaller, logical steps.
  - **Example:** To answer "If Alice has 3 apples and buys 2 more, how many does she have?", the model first adds 3 and 2, then states the answer.
  - **Benefits:**
    - Provides explanations or shows their thought process, making answers more transparent.
    - Better at tasks that require logic, calculation, or combining information from different sources.

Reasoning vs. Pattern-Matching in LLMs (cont.)

- **Examples**
  - "Which is heavier: a kilogram of feathers or a kilogram of iron?"

    A pattern-matching model may be misled by the association of "iron" with heaviness.

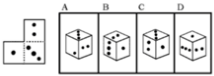
    A reasoning model will recognize that both weigh the same.

  - In logic puzzles, pattern-based models tend to contradict earlier premises

    Reasoning-aware models (e.g., using Chain-of-Thought prompting) remain consistent and logical.

Reasoning vs. Pattern-Matching in LLMs (cont.)

Still Terrible



Box folding: F
LLMs cannot solve box-folding problems and struggle with abstract figures and 3d spaces.

Abstract Map Navigation: D
LLMs cannot solve complex problems like route-planning in abstract 2d spaces.


Venn diagram fail


Supposed to be an unfolded box

Drawing diagrams: F
All tested models are comically bad at producing diagrams to exact, or even loose specifications.

Improving

Stacking objects: B+
Claude solved a puzzle GPT failed on a year ago.
Progress!

Real Map Navigation: C+
Models showed uneven but improving performance in multi-stop navigation.

Spatial reasoning in photos: B
Models seem to do better reasoning from photos of real scenes than from abstractions.

Venn Diagrams: B
Claude and GPT can reason from verbal descriptions of Venn diagrams.

Reasoning vs. Pattern-Matching in LLMs (cont.)

- **Empirical Separation: When Does Reasoning Matter?**
  - On simple factual questions, both model types may perform well.
  - For challenging tasks like GSM8K (grade-school math), symbolic reasoning, or logic puzzles, pattern-matching alone is not enough.
  - Success on these benchmarks improves dramatically when models are prompted to reason step by step (e.g., using Chain-of-Thought prompting).
  - *Example:* In multi-step math problems, models that show their work (reasoning-focused) achieve much higher accuracy than those that just guess the answer.
  - This highlights the need for explicit reasoning capabilities in advanced language models.

Following are some of the leading Large Reasoning Models (LRMs) as of 2024/2025:

- **OpenAI O1/O3**
  - **Architecture:** Dense Transformer, RLHF-optimized.
  - **Training Data Transparency:** Proprietary, limited details.
  - **Strengths:** High-quality reasoning, STEM, coding.
  - **Reasoning Approach:** Adjustable effort (Low/Med/High), step-by-step.
  - **Efficiency:** Adjustable reasoning mode, 200K context window.
  - **Best Use Case:** General AI, STEM research, API function calling.

Leading Large Reasoning Models (cont.)

- **DeepSeek R1**
  - **Architecture:** Mixture of Experts (MoE, 671B total, 37B active).
  - **Training Data Transparency:** Somewhat open (14.8T tokens).
  - **Strengths:** Expert coding & debugging, efficient MoE.
  - **Reasoning Approach:** Multi-expert collaboration, self-improvement.
  - **Efficiency:** High token efficiency (MoE, 312 tokens/sec).
  - **Best Use Case:** Enterprise coding assistant, code debugging.

- **Gemini 2.0 (Flash Thinking)**
  - **Architecture:** Multimodal Transformer (Text+Image+Speech).
  - **Training Data Transparency:** Proprietary, multimodal (text, images, APIs).
  - **Strengths:** Multimodal (images, text, speech), deep reasoning.
  - **Reasoning Approach:** Flash Thinking mode (transparent thought process).
  - **Efficiency:** 1M token context (experimental), optimized for speed.
  - **Best Use Case:** AI assistant (multimodal, research, planning).

Leading Large Reasoning Models (cont.)

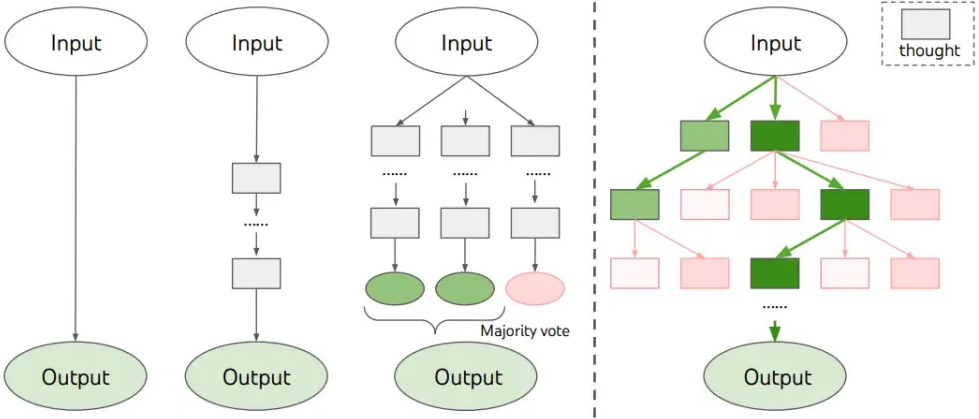
- **Claude 3.5**
  - **Architecture:** Dense Transformer, fine-tuned for dialogue & ethics.
  - **Training Data Transparency:** Partially open (some RLHF data available).
  - **Strengths:** Long-form dialogue, ethical AI, context memory.
  - **Reasoning Approach:** Contextual understanding, ethical guardrails.
  - **Efficiency:** Long context (200K), RLHF for alignment.
  - **Best Use Case:** Long-form assistant, decision-making support.

- **Codestral (Mistral)**
  - **Architecture:** Dense Transformer, code-focused.
  - **Training Data Transparency:** Proprietary, open benchmarks used.
  - **Strengths:** Cost-effective code generation, small but optimized.
  - **Reasoning Approach:** Optimized for coding efficiency & debugging.
  - **Efficiency:** High speed (small model, transformer optimized).
  - **Best Use Case:** Cost-effective coding LLM.

Leading Large Reasoning Models (cont.)

- **QwQ (Alibaba)**
  - **Architecture:** Dense Transformer, fine-tuned for reasoning.
  - **Training Data Transparency:** Limited transparency, strong math focus.
  - **Strengths:** Math, logical puzzles, chain-of-thought self-verification.
  - **Reasoning Approach:** Self-reflection, chain-of-thought.
  - **Efficiency:** Small model (32B), inference-efficient.
  - **Best Use Case:** Mathematical proofs, advanced reasoning.

Reasoning Techniques in LLMs

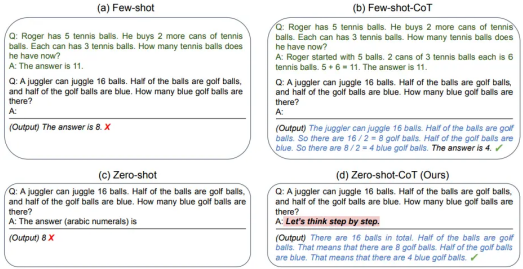

Input
Output
(a) Input-Output Prompting (IO)
Input
Output
(c) Chain of Thought Prompting (CoT)
Input
Majority vote
Output
(c) Self Consistency with CoT (CoT-SC)
Input
thought
Output
(d) Tree of Thoughts (ToT)

Chain-of-Thought Prompting (Recap)

- CoT prompting makes the model generate intermediate reasoning steps before the final answer, either by showing worked examples or by appending "Let's think step by step".
- For reasoning models this is the central mechanism: the techniques that follow make the intermediate steps longer, broader, or trained into the weights.
- Beyond basic CoT: automatic exemplar construction (Auto-CoT), search over reasoning paths (Tree-of-Thought), and sampling-based answer selection (Self-Consistency).

Few-shot vs Zero-shot CoT

- Few-shot CoT provides worked exemplars in the prompt (e.g. "If 6 times 4 equals 24, then what is 6 times 5?" followed by its reasoning steps); zero-shot CoT only appends an instruction such as "Let's think step by step".
- Few-shot buys accuracy at the cost of hand-crafting exemplars for each task; zero-shot avoids that cost but is weaker on hard problems. Auto-CoT aims for few-shot accuracy without the manual effort.



Comparison of Few-shot and Zero-shot CoT Prompting. Source: Zhang et al. (2022)

AUTOMATIC CHAIN OF THOUGHT PROMPTING IN LARGE LANGUAGE MODELS

Zhuosheng Zhang[†,*] **Aston Zhang**[‡], **Mu Li**[‡], **Alex Smola**[‡]
[†]Shanghai Jiao Tong University, [‡]Amazon Web Services

ABSTRACT

Large language models (LLMs) can perform complex reasoning by generating intermediate reasoning steps. Providing these steps for prompting demonstrations is called chain-of-thought (CoT) prompting. CoT prompting has two major paradigms. One leverages a simple prompt like "Let's think step by step" to facilitate step-by-step thinking before answering a question. The other uses a few manual demonstrations one by one, each composed of a question and a reasoning chain that leads to an answer. The superior performance of the second paradigm hinges on the hand-crafting of task-specific demonstrations one by one. We show that such manual efforts may be eliminated by leveraging LLMs with the "Let's think step by step" prompt to generate reasoning chains for demonstrations one by one, i.e., *let's think not just step by step, but also one by one*. However, these generated chains often come with mistakes. To mitigate the effect of such mistakes, we find that diversity matters for automatically constructing demonstrations. We propose an automatic CoT prompting method: Auto-CoT. It samples questions with diversity and generates reasoning chains to construct demonstrations. On ten public benchmark reasoning tasks with GPT-3, Auto-CoT consistently matches or exceeds the performance of the CoT paradigm that requires manual designs of demonstrations. Code is available at `https://github.com/amazon-research/auto-cot`

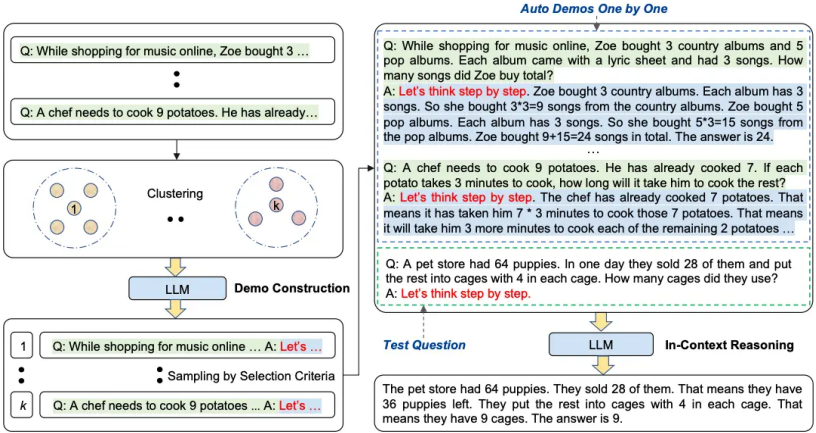
Auto-CoT Prompting (cont.)

- **What is Auto-CoT?**
  - An automated approach to generating Chain-of-Thought prompts.
  - Uses algorithms to create reasoning steps based on the problem context.
- **How it works:**
  - The system analyzes the problem and automatically generates a sequence of reasoning steps.
  - This can be done using techniques like reinforcement learning or heuristic methods.

Auto-CoT Prompting (cont.)

- **Benefits:**
  - Reduces the manual effort required to create effective CoT prompts.
  - Can adapt to a wide range of problems without needing specific examples.
- **Example:**
  - The system might take a problem like "If Alice has 3 apples and gives 2 to Bob, how many does she have left?" and automatically generate reasoning steps like "Start with 3 apples, subtract 2 apples given to Bob, and conclude with the remaining number of apples."

Auto-CoT Prompting (cont.)



Auto-CoT Prompting. Source: Zhang et al. (2022)

Auto-CoT Prompting (cont.)

Algorithm 1 Cluster

Require: A set of questions $\mathcal{Q}$ and the number of demonstrations k

Ensure: Sorted questions $\mathbf{q}^{(i)} = [q_1^{(i)}, q_2^{(i)}, \ldots]$ for each cluster i $(i = 1, \ldots, k)$

procedure CLUSTER($\mathcal{Q}$, k)
  for each question q in $\mathcal{Q}$ **do**
    Encode q by Sentence-BERT
  Cluster all the encoded question representations into k clusters
  for each cluster $i = 1, \ldots, k$ **do**
    Sort questions $\mathbf{q}^{(i)} = [q_1^{(i)}, q_2^{(i)}, \ldots]$ in the ascending order of the distance to the cluster center
  return $\mathbf{q}^{(i)}$ $(i = 1, \ldots, k)$

Auto-CoT Prompting (cont.)

Algorithm 2 Construct

Require: Sorted questions $\mathbf{q}^{(i)} = [q_1^{(i)}, q_2^{(i)}, \ldots]$ for each cluster i $(i = 1, \ldots, k)$, empty demonstration list $\mathbf{d}$

Ensure: Demonstration list $\mathbf{d} = [d^{(1)}, \ldots, d^{(k)}]$

procedure CONSTRUCT($\mathbf{q}^{(i)}, \ldots, \mathbf{q}^{(k)}$)
  for each cluster $i = 1, \ldots, k$ **do**
    for each question $q_j^{(i)}$ in $\mathbf{q}^{(i)}$ **do**
      Generate rationale $r_j^{(i)}$ and answer $a_j^{(i)}$ for $q_j^{(i)}$ using Zero-Shot-CoT
      if $q_j^{(i)}, r_j^{(i)}$ satisfy selection criteria **then**
        Add $d^{(i)} = [\text{Q: } q_j^{(i)}, \text{A: } r_j^{(i)} \circ a_j^{(i)}]$ to $\mathbf{d}$
        break
  return $\mathbf{d}$

Tree-of-Thought (ToT) Reasoning

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao
Princeton University

Dian Yu
Google DeepMind

Jeffrey Zhao
Google DeepMind

Izhak Shafran
Google DeepMind

Thomas L. Griffiths
Princeton University

Yuan Cao
Google DeepMind

Karthik Narasimhan
Princeton University

Abstract

Language models are increasingly being deployed for general problem solving across a wide range of tasks, but are still confined to token-level, left-to-right decision-making processes during inference. This means they can fall short in tasks that require exploration, strategic lookahead, or where initial decisions play a pivotal role. To surmount these challenges, we introduce a new framework for language model inference, "Tree of Thoughts" (ToT), which generalizes over the popular "Chain of Thought" approach to prompting language models, and enables exploration over coherent units of text ("thoughts") that serve as intermediate steps toward problem solving. ToT allows LMs to perform deliberate decision making by considering multiple different reasoning paths and self-evaluating choices to decide the next course of action, as well as looking ahead or backtracking when necessary to make global choices. Our experiments show that ToT significantly enhances language models' problem-solving abilities on three novel tasks requiring non-trivial planning or search: Game of 24, Creative Writing, and Mini Crosswords. For instance, in Game of 24, while GPT-4 with chain-of-thought prompting only solved 4% of tasks, our method achieved a success rate of 74%. Code repo with all prompts: `https://github.com/princeton-nlp/tree-of-thought-llm`.

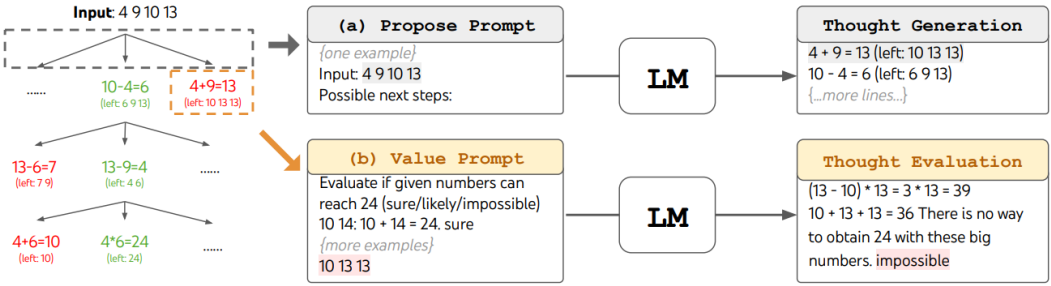
Tree-of-Thought (ToT) Reasoning (cont.)

- **What is ToT?**
  - A reasoning technique that structures the thought process as a tree, allowing for exploration of multiple reasoning paths.
  - Each node in the tree represents a step in the reasoning process, and branches represent different possible paths or decisions.
- **How it works:**
  - The model generates a tree structure where each branch represents a different reasoning path.
  - The model can explore multiple paths simultaneously, evaluating the outcomes of each.

Tree-of-Thought (ToT) Reasoning (cont.)

- **Benefits:**
  - Allows for more complex reasoning by considering multiple possibilities at once.
  - Helps the model avoid getting stuck in a single line of reasoning, improving problem-solving capabilities.
- **Example:**
  - For a problem like "If Alice has 3 apples and gives some to Bob, how many does she have left?", the model might create a tree with branches for different amounts given to Bob (e.g., 1, 2, or 3 apples) and explore the outcomes of each branch.

Tree-of-Thought (ToT) Reasoning (cont.)



ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation. Source: Yao et al. (2023)

Algorithm 1 ToT-BFS$(x, p_\theta, G, k, V, T, b)$

Require: Input x, LM p_θ, thought generator $G()$ & size limit k, states evaluator $V()$, step limit T, breadth limit b.

$S_0 \leftarrow \{x\}$

for $t = 1, \cdots, T$ **do**

$S'_t \leftarrow \{[s, z] \mid s \in S_{t-1}, z_t \in \mathrm{G}(p_\theta, s, k)\}$

$V_t \leftarrow V(p_\theta, S'_t)$

$S_t \leftarrow \arg\max_{S \subset S'_t, |S|=b} \sum_{s \in S} V_t(s)$

end for

return $G(p_\theta, \arg\max_{s \in S_T} V_T(s), 1)$

Algorithm 2 ToT-DFS$(s, t, p_\theta, G, k, V, T, v_{th})$

Require: Current state s, step t, LM p_θ, thought generator $G()$ and size limit k, states evaluator $V()$, step limit T, threshold v_{th}

if $t > T$ **then** record output $G(p_\theta, s, 1)$
end if
for $s' \in G(p_\theta, s, k)$ **do** ▷ sorted candidates
  if $V(p_\theta, \{s'\})(s) > v_{thres}$ **then** ▷ pruning
    DFS$(s', t+1)$
  end if
end for

Tree-of-Thought (ToT) Reasoning (cont.)

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	**74%**
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

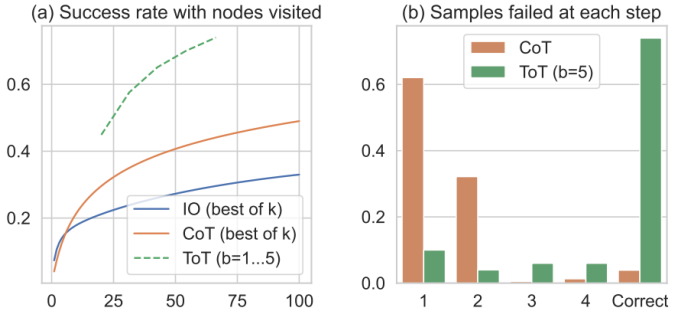


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

Tree-of-Thought (ToT) Reasoning (cont.)

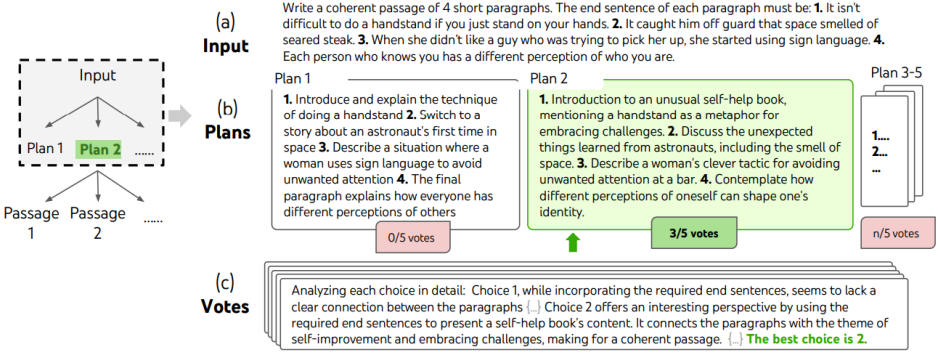


Figure 4: A step of deliberate search in a randomly picked Creative Writing task. Given the input, the LM samples 5 different plans, then votes 5 times to decide which plan is best. The majority choice is used to consequently write the output passage with the same sample-vote procedure.

Tree-of-Thought (ToT) Reasoning (cont.)

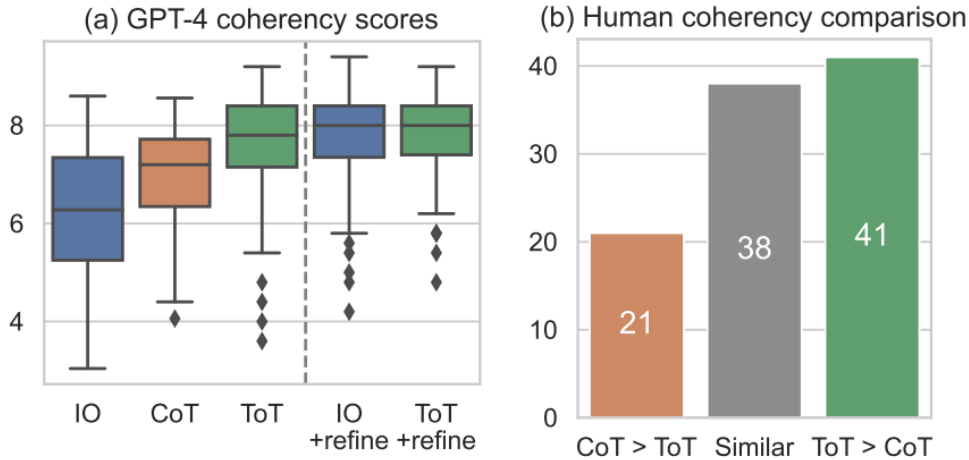


Figure 5: Creative Writing results.

Self-Consistency (SC)

- Self-consistency samples several reasoning paths for the same prompt, then keeps the answer the paths agree on.
- The figures below give the empirical picture from Wang et al. (2022): consistent gains across arithmetic and commonsense benchmarks, growing with the number of sampled paths, at a linear increase in inference cost.
- Sampling many paths at inference time is an early form of test-time compute scaling, the same trade later used by dedicated reasoning models.

Self-Consistency Improves Chain of Thought Reasoning in Language Models

Xuezhi Wang[†‡] **Jason Wei**[†] **Dale Schuurmans**[†] **Quoc Le**[†] **Ed H. Chi**[†]
Sharan Narang[†] **Aakanksha Chowdhery**[†] **Denny Zhou**[†§]
[†]Google Research, Brain Team
[‡]xuezhiw@google.com, [§]dennyzhou@google.com

Abstract

Chain-of-thought prompting combined with pre-trained large language models has achieved encouraging results on complex reasoning tasks. In this paper, we propose a new decoding strategy, *self-consistency*, to replace the naive greedy decoding used in chain-of-thought prompting. It first samples a diverse set of reasoning paths instead of only taking the greedy one, and then selects the most consistent answer by marginalizing out the sampled reasoning paths. Self-consistency leverages the intuition that a complex reasoning problem typically admits multiple different ways of thinking leading to its unique correct answer. Our extensive empirical evaluation shows that self-consistency boosts the performance of chain-of-thought prompting with a striking margin on a range of popular arithmetic and commonsense reasoning benchmarks, including GSM8K (+17.9%), SVAMP (+11.0%), AQuA (+12.2%), StrategyQA (+6.4%) and ARC-challenge (+3.9%).

Self-Consistency (SC) (cont.)

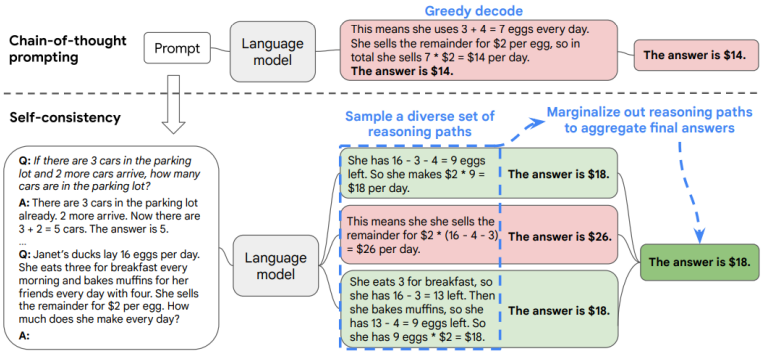


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the "greedy decode" in CoT prompting by sampling from the language model's decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

Source: Wang et al. (2022)

Self-Consistency (SC) (cont.)

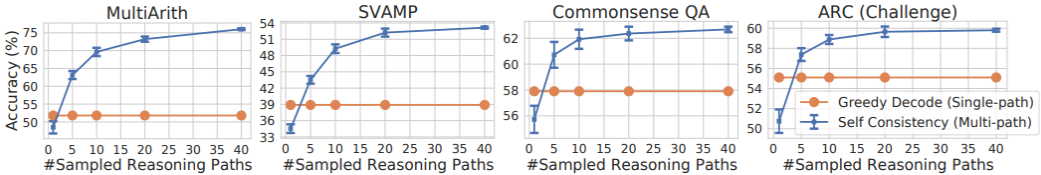


Figure 2: Self-consistency (blue) significantly improves accuracy over CoT-prompting with greedy decoding (orange) across arithmetic and commonsense reasoning tasks, over LaMDA-137B. Sampling a higher number of diverse reasoning paths consistently improves reasoning accuracy.

Self-Consistency (SC) (cont.)

	GSM8K	StrategyQA
Question	*Henry made two stops during his 60-mile bike trip. He first stopped after 20 miles. His second stop was 15 miles before the end of the trip. How many miles did he travel between his first and second stops?*	*Yes or no: Is Albany, Georgia the most populous US Albany?*
Greedy Decode	He traveled 60 miles in total. He stopped after 20 miles, so he traveled 60 - 20 = 40 miles between the first and second stops. The answer is 40.	Albany, Georgia is the most populous US Albany. Thus, the answer is yes.
Sampled Path 1	Henry travelled a total of 60 miles. His first stop was 20 miles in, and his second stop was 15 miles before the end. So between his first and second stops he travelled 60 - 20 -15 = 25 miles. The answer is 25.	The most populous US Albany is Albany, New York. Thus, Albany, Georgia is not the most populous US Albany. So the answer is no.
Sampled Path 2	He made two stops during a 60-mile trip. The first was 20 miles into the trip. The second was 15 miles before the end of the trip. This means the second stop was 60 - 15 = 45 miles into the trip. Since he made the stops in order, the second stop must have been 45 - 20 = 25 miles after the first stop. The answer is 25.	Albany, Georgia has a population of about 88,000. Albany, New York has a population of about 95,000. Thus, Albany, Georgia is not the most populous US Albany. So the answer is no.

Table 4: Examples where self-consistency helps repair the errors over greedy decode, on PaLM-540B. Two sampled reasoning paths that are consistent with the ground truth are shown.

Self-Consistency (SC) (cont.)

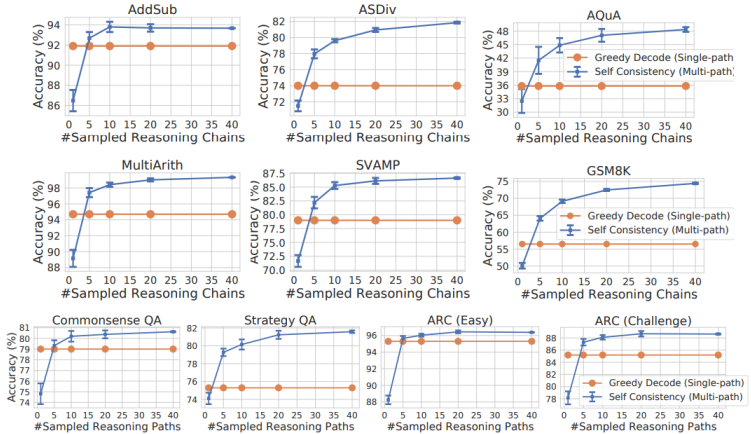
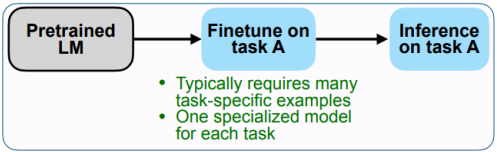


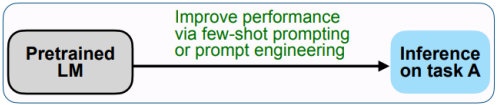
Figure 8: Self-consistency (blue) significantly improves accuracy across various arithmetic and commonsense reasoning tasks, over PaLM-540B. Sampling a higher number of diverse reasoning paths consistently helps reasoning accuracy.

Instruction Tuning for Reasoning Tasks

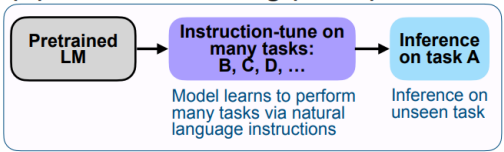
(A) Pretrain–finetune (BERT, T5)



(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



Instruction Tuning for Large Language Models: A Survey

Shengyu Zhang♠, Linfeng Dong♠, Xiaoya Li♠, Sen Zhang♠

Xiaofei Sun♠, Shuhe Wang♠, Jiwei Li♠♣, Runyi Hu♠
Tianwei Zhang▲, Fei Wu♠ and Guoyin Wang◆

Abstract

This paper surveys research works in the quickly advancing field of instruction tuning (IT), a crucial technique to enhance the capabilities and controllability of large language models (LLMs). Instruction tuning refers to the process of further training LLMs on a dataset consisting of (INSTRUCTION, OUTPUT) pairs in a supervised fashion, which bridges the gap between the next-word prediction objective of LLMs and the users' objective of having LLMs adhere to human instructions. In this work, we make a systematic review of the literature, including the general methodology of IT, the construction of IT datasets, the training of IT models, and applications to different modalities, domains and application, along with analysis on aspects that influence the outcome of IT (e.g., generation of instruction outputs, size of the instruction dataset, etc). We also review the potential pitfalls of IT along with criticism against it, along with efforts pointing out current deficiencies of existing strategies and suggest some avenues for fruitful research. ‡ ‡ ‡ ‡

Instruction Tuning (cont.)

- **What is Instruction Tuning?**
  - Fine-tuning LLMs on a dataset of instruction-response pairs.
  - Enhances the model's ability to follow instructions and perform tasks.
- **Why is it important?**
  - Improves model performance on diverse tasks.
  - Enables better generalization to unseen tasks.

Instruction Tuning (cont.)

Why Instruction Tune LLMs?

- Pre-trained LLMs are optimized for next-word prediction, not for following instructions or engaging in conversations.
- Without instruction tuning, LLMs may generate text that is linguistically correct but not aligned with user intent.
- Instruction tuning uses supervised learning on instruction-response pairs, teaching the model to generate helpful and relevant outputs for user prompts.
- Fine-tuning is much more efficient than training a large model from scratch and can be performed with less data and compute.
- Instruction tuning bridges the gap between generic language modeling and practical, task-oriented use cases, making LLMs more useful and predictable.

Instruction Tuning (cont.)

Summary of Instruction Tuning

- **Definition:** Fine-tuning LLMs on diverse instruction-response pairs.
- **Key Elements:**
  - *Instruction:* Task description in natural language.
  - *Additional Information:* Optional context or input.
  - *Desired Output:* Target response for evaluation.
- **Benefits:**
  - Improves ability to follow instructions.
  - Reduces need for prompt engineering and few-shot examples.
  - Enhances generalization to unseen tasks.
- **Applications:**
  - Translation, question answering, reading comprehension, NLI.
  - Broad improvements across reasoning and practical tasks.

Examples of Instruction Tuning in Practice

- **FLAN-T5:**
  - Instruction-tuned T5 models show strong improvements on reasoning benchmarks such as DROP (reading comprehension), ARC (science questions), and MultiRC (multi-sentence reasoning).
  - For example, when given instructions like "Answer the following science question" or "Read the passage and answer the question," FLAN-T5 produces more accurate and relevant responses.
  - *Example:*
    - **Instruction:** "Read the passage and answer: What did the boy do after school?"
    - **Passage:** "The boy finished his homework and went to play soccer."
    - **FLAN-T5 Output:** "He went to play soccer."

Instruction Tuning (cont.)

- **FLAN-MoE:**
  - Mixture-of-Experts (MoE) models, when instruction-tuned, outperform larger dense models like PaLM on reasoning tasks, while using less computation.
  - FLAN-MoE can handle a wide range of instructions, such as translation, summarization, and logical reasoning, with high accuracy.
  - *Example:*
    - **Instruction:** "Translate this sentence to French: The cat is sleeping."
    - **FLAN-MoE Output:** "Le chat dort."

Instruction Tuning (cont.)

Why Does Instruction Tuning Work?

- Models learn to **follow instructions step by step**, rather than just predicting the next word or memorizing answers.
- Encourages **logical reasoning**: Models are guided to explain their answers or show their work, which helps in tasks like math, logic puzzles, and multi-step questions.
- **Generalization:** Instruction-tuned models can handle new types of questions or tasks they have not seen before, simply by following the provided instruction.
- **Variety of Examples:**
  - *Math:* "Solve: What is 15 divided by 3?" $\rightarrow$ "5"
  - *Reasoning:* "Is a whale a mammal or a fish? Explain your answer." $\rightarrow$ "A whale is a mammal because it gives birth to live young and breathes air."
  - *Summarization:* "Summarize the following paragraph in one sentence."

Instruction Tuning (cont.)

Instruction Tuning vs. Multi-Task Fine-Tuning (FLAN Ablation Study)

- **Research Question:** Are the benefits of instruction tuning due to the instructions themselves, or just from fine-tuning on multiple tasks?
- **Ablation Setups:**
  - *No Template:* Only input-output pairs, e.g., input: "the dog runs", output: "le chien court".
  - *Dataset Name:* Input prepended with task and dataset, e.g., "[Translation: WMT 14 to French] The dog runs".
  - *FLAN Instructions:* Full natural language instruction, e.g., "Please translate this sentence to French: 'The dog runs.' "
- **Results:**
  - Instruction-tuned models achieved over 18% higher accuracy than "no template" and over 8% higher than "dataset name" on zero-shot tasks.
  - **Conclusion:** Explicit instructions are crucial for improving zero-shot generalization, not just multi-task fine-tuning.

Instruction Tuning (cont.)

Chain-of-Thought (CoT) Fine-Tuning

- **How is it done?** This can be achieved by:
  - Few-shot prompting with exemplars that demonstrate sequential reasoning.
  - Appending phrases like "think step by step" to prompts.
- **Why is it important?**
  - CoT prompting significantly enhances zero-shot performance on arithmetic, symbolic, and logical reasoning tasks.
  - Research (Chung et al., 2022) shows that instruction tuning without CoT tasks degrades performance on CoT evaluations.
  - Including CoT tasks in instruction tuning improves performance across all evaluations.
- **Key Insight:** Fine-tuning on CoT tasks—both with and without few-shot exemplars—enables models to better perform step-by-step reasoning, rather than jumping to linguistically plausible answers.

Instruction Tuning (cont.)

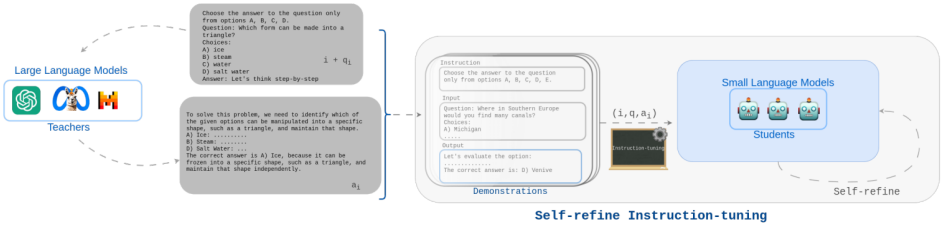


Figure 1: In Self-refine Instruction-tuning, the Demonstrations delivered by teacher models are used to align reasoning abilities in a teacher-student setting. Following the transference of step-wise reasoning knowledge via instruction tuning, the students Self-refine their abilities with the support of Direct Preference Optimization methods.

Instruction tuning process. Source: Ranaldi et al. (2024)

Instruction Tuning (cont.)

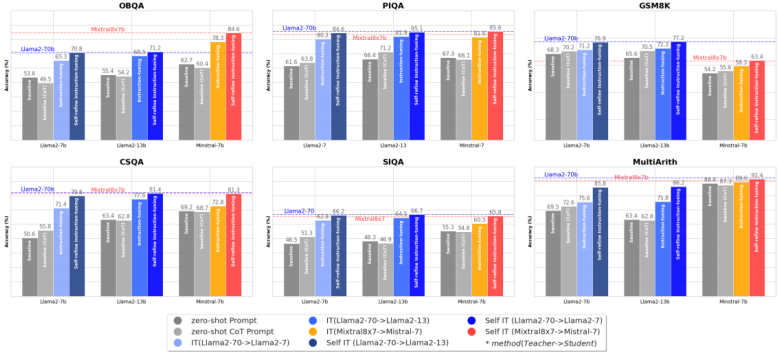


Figure 2: Accuracies (%) on benchmarks (Section 3.1) before Instruction-tuning (i.e., `Baselines` and `Baseline CoT`), after Instruction-tuning (IT) performed on Demonstrations delivering CoT and finally behind the `Self-refine Instruction-tuning` phase (Self IT). In particular, the models were instructed via Demonstrations delivered by in-family LLMs (as described in the legend, we use the notation *method(Teacher->Student)*).

Source: Ranaldi et al. (2024)

Instruction Tuning (cont.)

Prompt for task: OBQA, CSQA, PIQA, SIQA	*Prompt for task:* GSM8k, MultiArith
Choose the answer to the question only from options A, B, C, [...]. **Question:** <Question> **Choices:** A) <Option1> B) <Option2> C) <Option3> .... **Answer:**	**Answer the following mathematical question with numerical solution.** **Question:** <Question> **Answer:**

Table 2: Example of input-prompt for multiple-choices (left) and mathematical (right) question-answering benchmarks.

Prompt for task: OBQA, CSQA, PIQA, SIQA	*Prompt for task:* GSM8k, MultiArith
Choose the answer to the question only from options A, B, C, [...]. **Question:** <Question> **Choices:** A) <Option1> B) <Option2> C) <Option3> .... **Answer: Let's think step by step**	**Answer the following mathematical question with numerical solution.** **Question:** <Question> **Answer: Let's think step by step**

Table 3: Example **Zero-shot CoT** of input-prompt for multiple-choices (left) and mathematical (right) question-answering benchmarks (approach used in this work).

Instruction Tuning (cont.)

Instruction

```
Choose the answer to the question only from options A, B, C, D.
```

Input

```
Question: Which form can be made into a triangle?
Choices:
A) ice
B) steam
C) water
D) salt water
Answer: Let's think step-by-step
```

Output (that is the Answer produced by teacher model)

```
Answer:
A) ice: Ice is a solid form of water. In solid form, substances can be molded or shaped into
various forms, including triangles. If you had a block of ice, you could carve or mold it
into a triangular shape.
B) steam: Steam is the gaseous form of water. Being a gas, it doesn't have a fixed shape. You
can't shape steam into a triangle without containing it within some form of solid boundary,
and even then, the steam itself isn't triangular - the container is.
C) water: Water is a liquid that takes the shape of its container. You can pour water into
a triangular container, and while the volume inside would be triangular, the water itself
isn't inherently triangular.
D) salt water: This is essentially the same as option C, but with salt dissolved in it. Being
a liquid, it takes the shape of its container.
Given these considerations, the only substance among the options that can be shaped into a
triangle by itself is A) Ice.
```

Table 6: Example of Demonstration produced by the GPT-3.5 answer on an Open Book Question Answering benchmark instance. The structure is composed by: **Instruction**, **Input** and **Output**.

Source: Ranaldi et al. (2024)

Challenges and Limitations of Instruction Tuning

- **Dataset Quality and Diversity:**
  - Large, high-quality datasets are costly to create.
  - Reliance on few open-source datasets may reduce diversity.
- **Bias Propagation:**
  - Proprietary LLMs used for instruction generation can spread their biases to open-source models.
- **Evaluation Concerns:**
  - Evaluation often uses proprietary models, which may not reveal all biases or limitations.

Instruction Tuning (cont.)

- **Superficial Learning:**
  - Smaller models may mimic style without true reasoning.
  - Gains may reflect pattern matching, not understanding.
- **Overfitting to Training Tasks:**
  - Evaluation tasks too similar to training can overstate improvements.
- **Key Insight:**
  - Improving base model quality is more impactful than just imitating proprietary systems.

Limitations of Large Reasoning Models

- **Verbose outputs:** CoT (Chain-of-Thought) and ToT (Tree-of-Thought) methods generate long outputs.
- May confuse end-users due to extensive reasoning traces.
- **Prompt sensitivity:** Small changes in prompt wording lead to large changes in output.
- Requires careful prompt engineering.
- **Computation overhead:** Especially with self-consistency sampling or ToT trees.
- Increases inference cost and time.
- **Example:** CoT prompting works well on GSM8K (a math dataset), but fails without careful prompt tuning.
- Shows sensitivity to specific prompt design.

Summary

- **Large reasoning models** differ from pattern-matching LLMs: they spend test-time compute on explicit multi-step reasoning traces.
- **Prompting techniques** (Chain-of-Thought, Tree-of-Thought, Self-Consistency) elicit reasoning from capable models without any training.
- **Instruction tuning** (FLAN and successors) teaches models to follow task descriptions and generalize to unseen tasks; CoT fine-tuning bakes reasoning into the weights.
- The leading systems (o1/o3, DeepSeek-R1, Gemini Flash Thinking, QwQ) combine these ingredients with RL post-training.
- Costs remain: verbose traces, prompt sensitivity, and inference compute overhead.

References & Further Reading

Prompting for reasoning

- Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903.
- Kojima, T., et al. (2022). *Large Language Models are Zero-Shot Reasoners*. arXiv:2205.11916.
- Zhang, Z., et al. (2022). *Automatic Chain of Thought Prompting in Large Language Models*. arXiv:2210.03493.
- Yao, S., et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. arXiv:2305.10601.
- Wang, X., et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. arXiv:2203.11171.

Instruction tuning

- Wei, J., et al. (2022). *Finetuned Language Models are Zero-Shot Learners* (FLAN). arXiv:2109.01652.

References & Further Reading (cont.)

- Chung, H. W., et al. (2022). *Scaling Instruction-Finetuned Language Models*. arXiv:2210.11416.

Reasoning models

- OpenAI (2024). *Learning to Reason with LLMs*. openai.com.
- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948.
- Financial Times (2025). *Transcript: The story behind DeepSeek's breakthrough*. ft.com.